

読む技術・書く技術・伝える技術

15年続けて分かった持続可能なオープンソース開発

azu (@azu_re)

YAPC::Fukuoka 2025

自己紹介

azu

- GitHub: @azu
- 2011年から14年間オープンソース活動を継続
- 3つのプロジェクトを運営中



今日話したいこと

技術を使って、持続可能性を実現する

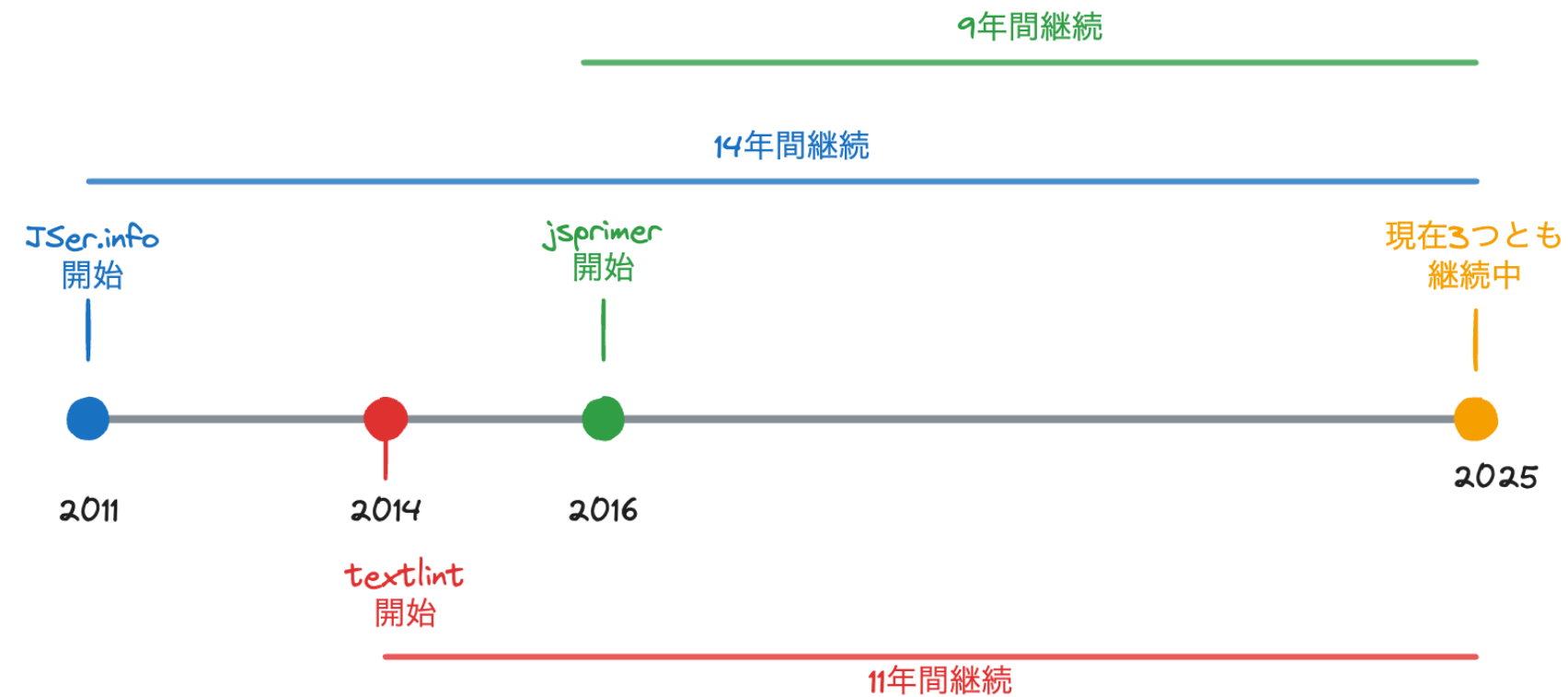
14年間JSer.infoを続け

11年間textlintを続け

9年間jsprimerを続けてきた

その方法をお話しします

3つのプロジェクトのタイムライン



共通点：

- 長期間継続している
- 今も活発に更新されている
- 複数のコントリビューターが参加

それぞれのテーマ

- **JSer.info:** 読む技術（情報収集と発信）
- **textlint:** 書く技術（品質の自動化）
- **jsprimer:** 伝える技術（協働と設計）

問い: なぜ15年も続けられたのか？

これらのプロジェクトの特徴

一つのアウトプットでは目的が達成できない

- JSer.info: 継続的な情報発信でエコシステムを把握
- textlint: あらゆる言語のlint、ルールとエコシステムの構築
- jsprimer: JavaScriptの進化に追従し続ける

ある程度長くやらないと達成できない目的

長期的な目的を持っている

これらのプロジェクトは
作ったもの（アウトプット）ではなく
実際の影響（アウトカム）を目指している

アウトプット vs アウトカム

アウトカムが評価されるまでには時間がかかる

アウトプット

作ったもの

(記事、ツール、書籍)

アウトカム

実際の影響・成果

(信頼、エコシステム、教育)

この抽象的な区別がOSSで特に重要になる理由を次で具体化します。

オープンソースにおけるアウトカム

時間をかけて現れるもの

- ツールの普及と影響
- コミュニティの形成
- 知識の蓄積と共有
- エコシステムの成長

これらはすべて、時間をかけないと現れない

JSer.infoの例

- **1年目:** 個人の情報整理
- **5年目:** JavaScript情報の参考元として認知
- **10年目:** JSer.info Policyの明文化、10年分の構造化データを蓄積
- **14年目:** AI時代のMCP統合で新しい価値

継続しないと、アウトカムにたどり着けない

多くのオープンソースプロジェクトの現実

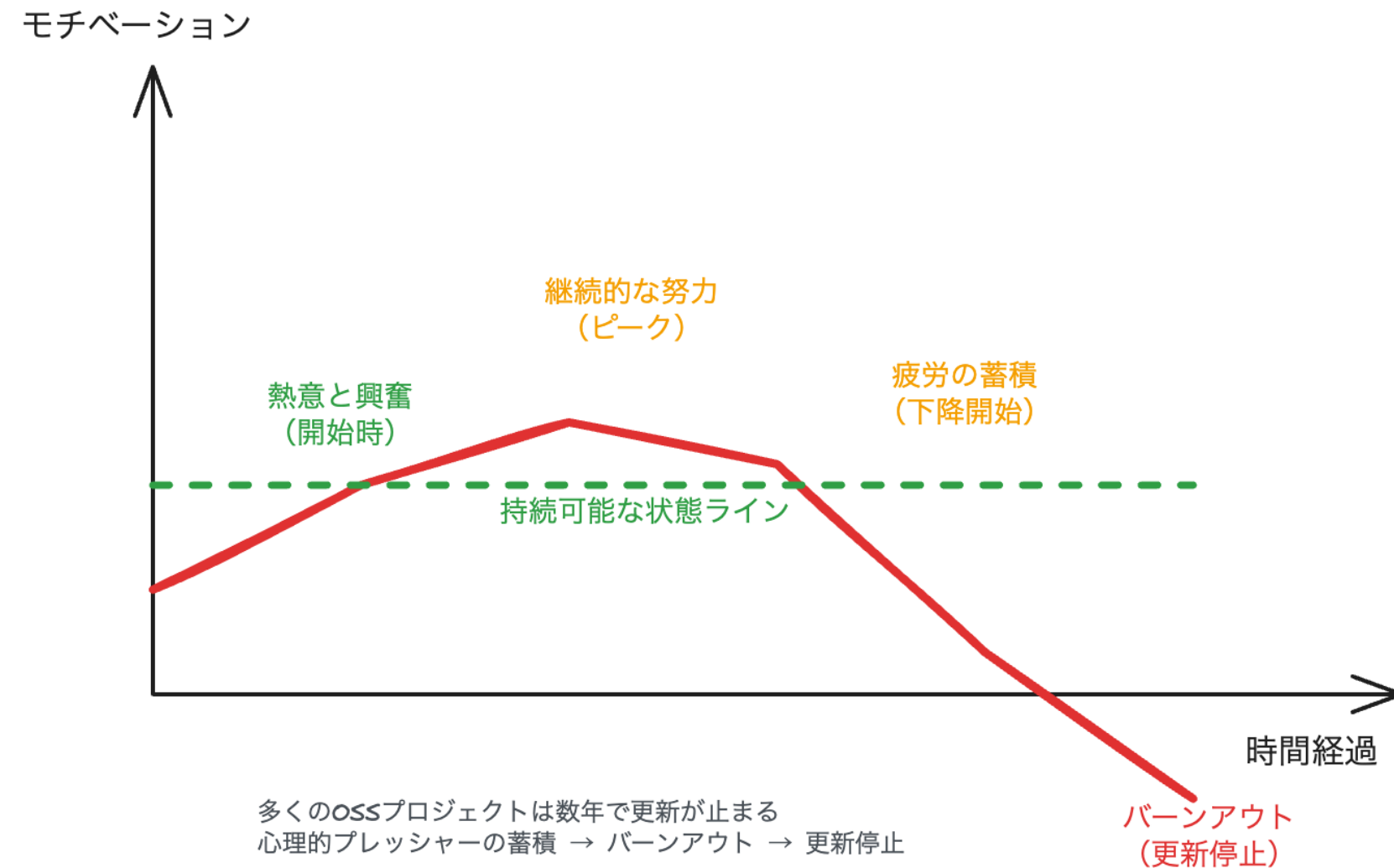
成功の裏側には、同じ期間を走り切れない多数のプロジェクトがあります。

- 数年で更新が止まる
- メンテナーの燃え尽き
- 心理的負担の増大

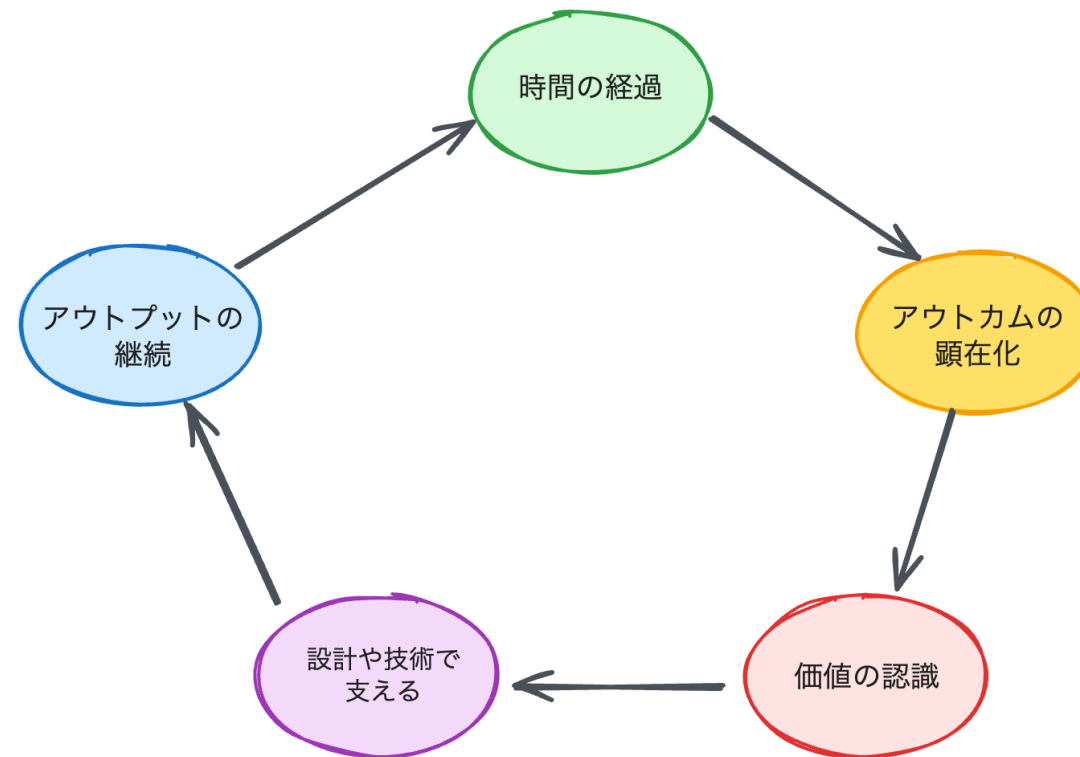
なぜ燃え尽きるのか？

→ 心理的プレッシャーの蓄積

このギャップを埋める鍵が『持続可能性の設計』です。



持続可能性が必要な理由



核心の問い：

どうやって15年間、燃え尽きずに継続できたのか？

この循環を支える具体的な設計指針が次の二軸です。

デスマ

技術的依存を増やし

心理的依存を減らす

技術的依存とは

定義：

- 自動化できる部分は自動化する
- ツールやシステムを育てる
- 再現可能なプロセスを構築する
- データを蓄積する

特徴：

- スケールする
- 疲れない
- 複利効果を生む
- 他者も利用できる

心理的依存とは

定義：

- 「自分がやらなければ」というプレッシャー
- 完璧主義
- 義務感

特徴：

- スケールしない
- 疲れる
- 燃え尽きにつながる

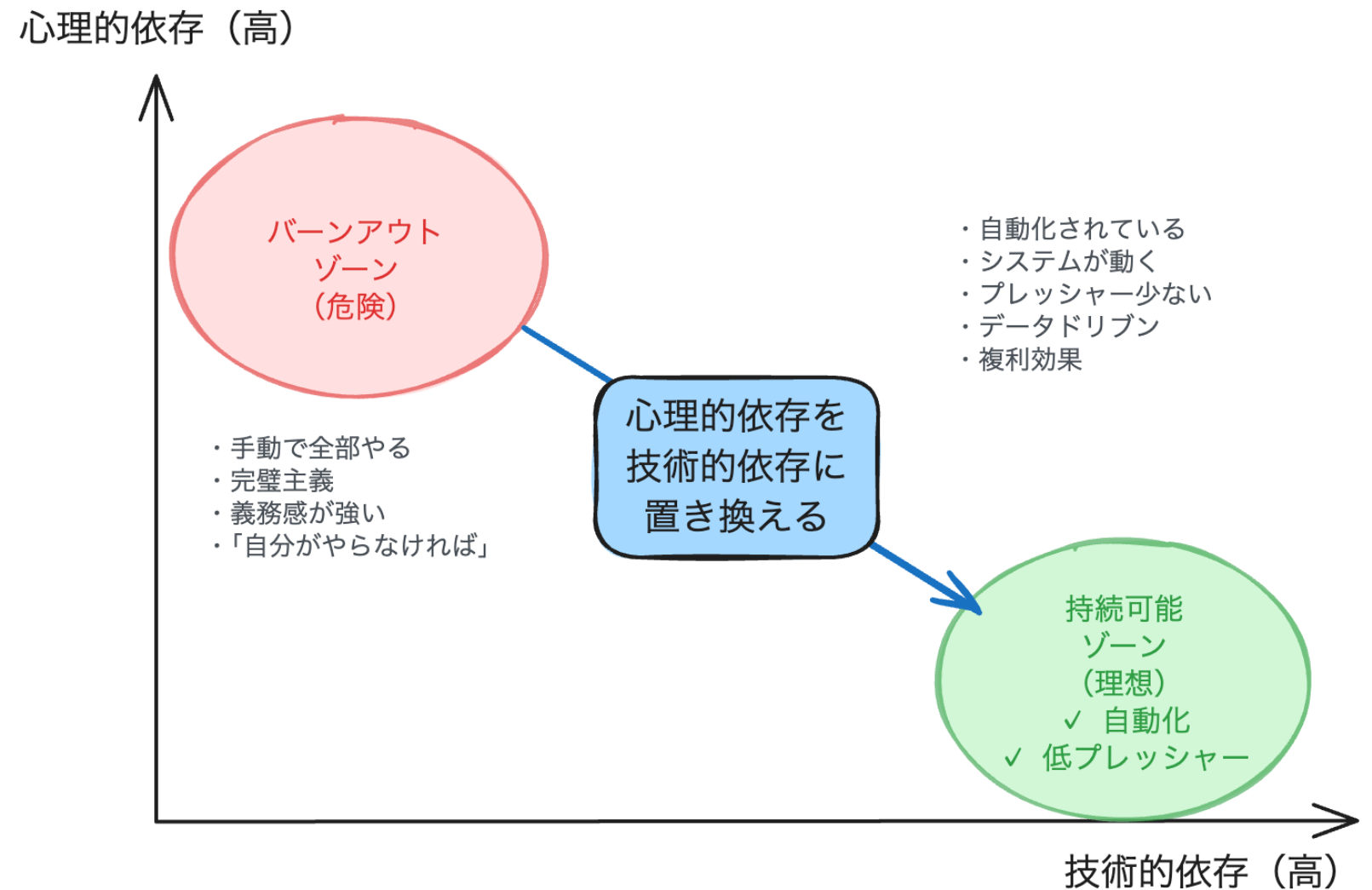
なぜこのバランスが重要か

技術的依存が高い

→ 長期的に継続できる

心理的依存が高い

→ バーンアウトのリスク



今日の構成

1. **JSer.info** - 読む技術

→ データドリブン、効率性vs効果性

2. **textlint** - 書く技術

→ No core rules、エコシステム

3. **jsprimer** - 伝える技術

→ Living Standard、既知→未知

Part 1

読む技術

JSer.info

JSer.info の本質

2011年1月16日創刊

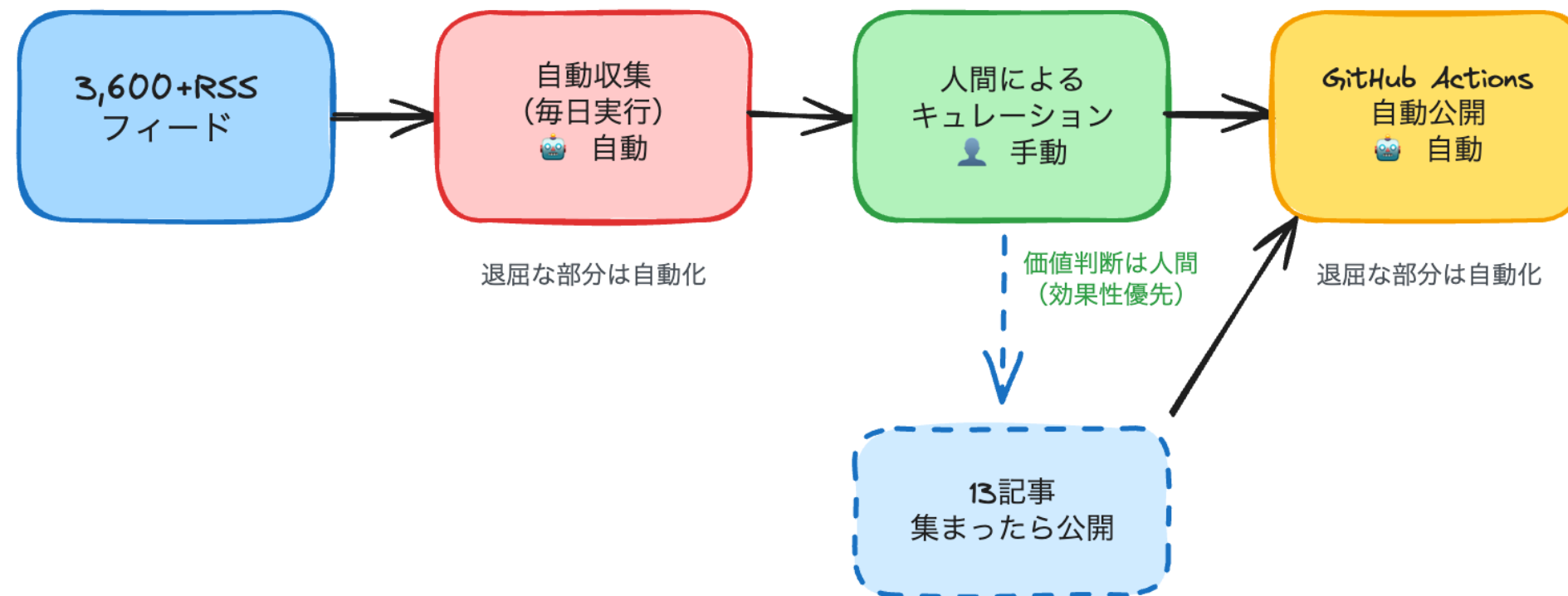
- 750記事、14年継続
- 12,429サイト紹介
- 一度も欠かさず週刊配信

整理されたデータである「情報」を伝えること

JSer.info Screenshot

情報収集システムの全体像

JSer.info 情報収集システムの全体像



なぜ完全自動化しないのか？

→ 効率性より効果性を優先

データドリブンな公開基準

✕ 従来

「毎週月曜日に公開」

→ 書けない週のプレッシャー

✓ JSer.info

「13記事集まったら公開」

→ 品質でリリース判断

結果: 14年間の継続

効率性 vs 効果性

効率性

- 物事を正しく行う
- 完全自動クロール

効果性

- 正しいことをする
- 人間によるキュレーション

JSer.infoの選択: 効果性を優先

退屈な部分だけ自動化

自動化するもの：

- 3,600フィードの収集（退屈）
- カテゴリ分類（退屈）
- GitHub Actions公開（退屈）

手動で残すもの：

- 価値判断（本質）
- 文脈の理解（本質）

JSer.info Policy

「技術的な嘘はつかない」

使わない言葉：

"is Dead"、最強、熱い

使う言葉：

代替方法、特徴、比較

感情極性値 -0.216（ほぼ中立）

「紹介」ではなく「知ってもらおう」

紹介

情報を提示して終わり

知ってもらおう

読者が元サイトにアクセス

JavaScriptエコシステム全体を
理解

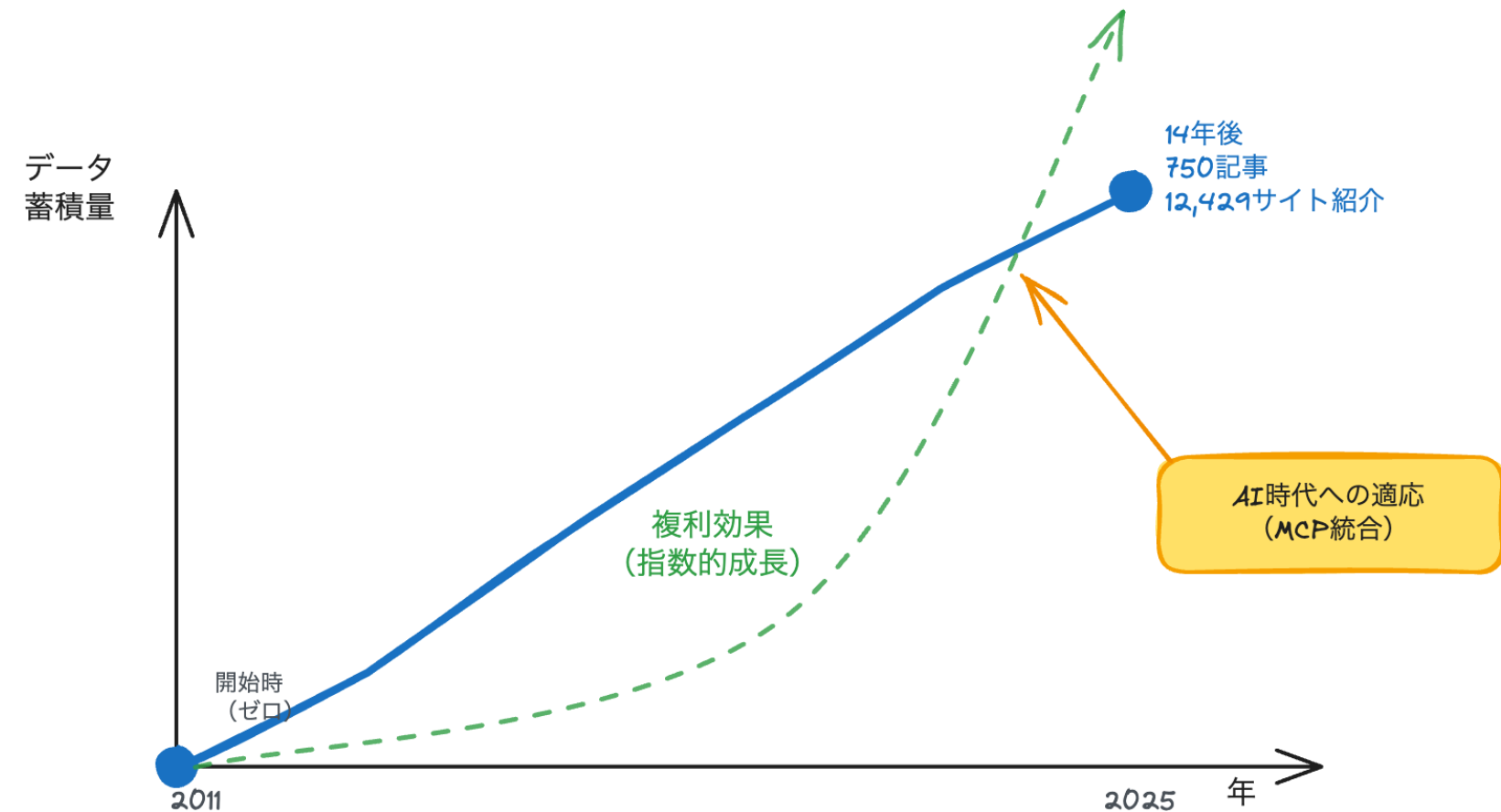
**JSer.infoは目的地ではなく、
導線**

技術的依存を育てる - 複利効果

14年間継続の結果：

- 14年分のデータ蓄積
- 構造化されたJSON
- AI時代への適応（MCP統合）

複利効果：過去の蓄積が未来の価値を生む



複利効果の具体例

- **2014年：textlint統合**
 - JSer.info記事の品質管理に使用
 - 使っているから改善する、改善したから使い続ける
- **2015年: Datasetと統計ライブラリを公開**
 - データセットをプログラムから利用可能に
- **2015年: JSer.info Trends**
 - 14年分のタグデータからトレンド可視化
 - エコシステムの変化を追跡
- **2022年: Product Name API**
 - 12,429サイトのデータからプロダクト名を検索
- **2022年: JSer.info Watch List公開**
 - 情報源をまとめたRSSを公開
- **2025年：JSer.info MCP Server**
 - 14年分のデータセットをClaude Desktopから検索可能に
 - AIがJavaScript情報の文脈を理解して回答

もし途中で辞めていたら、これらは存在しなかった

JSer.infoまとめ

技術的依存を増やした：

- データ基準（13記事で公開判断）
- 局所自動化（3,600フィード収集、GitHub Actions）
- 蓄積再利用（14年分のJSON、MCP統合）

心理的依存を減らした：

- 週次固定締切（毎週月曜のプレッシャー）
- 完璧主義（品質より継続）

Part 2

書く技術

textlint

読むから書くへの移行

読む技術の洞察が書く技術の改善をどう促したかを次で示します。

きっかけ：

- JSer.infoで情報を読み続けていた
- 「読むだけでは物足りない」
- 「自分でも書いてみよう」

2014年3月：Promise本の執筆開始

書くことで気づいたこと

書くと、読む量が増える

なぜ？

- 調べる（情報を読む）
- 検証する（ドキュメントを読む）
- 自分が書いたものを読み返す

書くことは、読むことの延長

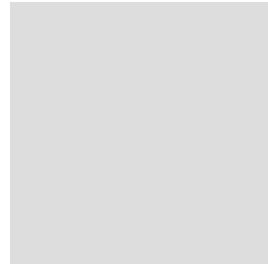
書く中で見つけた課題

Promise本執筆時の問題：

- 表記揺れが気になる
- 「JavaScript」 vs 「Javascript」
- 「コールバック」 vs 「callback」

解決策：ツールを作ろう

textlint誕生の5日間



Dec 24-30 2014 Timeline

5日間で作った理由：

自分の問題を解決したかった

「使うものを作る/使うために作る」

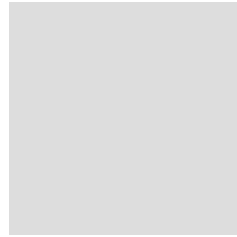
textlintの本質

「自然言語には正解がないため、
拡張の柔軟性を持つこと」

コア原則：

- 検証できることだけを自動化
- ルールの透明性
- 説明可能性

AST（抽象構文木）ベースの処理



Markdown to AST to Rules

正規表現ではなく構文解析

→ 文脈を理解した検証

No core rules戦略

✖ 従来のLinter

インストール → デフォルトルール

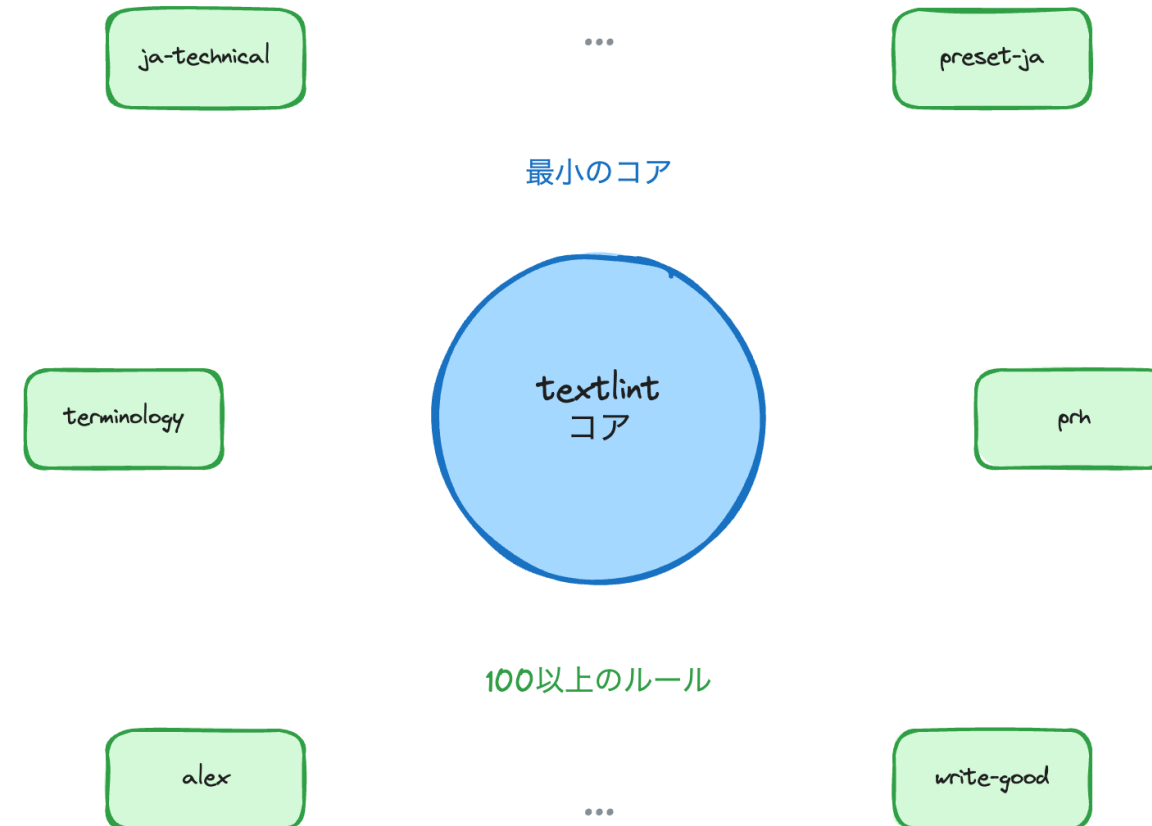
→ すぐ使える

✔ textlint

インストール → 何も起きない
→ ルール選択

「楽に使える」より「適切に
使える」

最小コア、豊かなエコシステム



結果：

週85,000ダウンロード

100以上のコミュニティルール

AI時代への適応 - MCP統合

2025年1月：textlint v14.8.0

```
npx textlint --mcp
```

- AIアシスタントから直接呼び出し可能
- VS Code、Cursor、Claude Code統合

11年前のツールが、AI時代でも価値を持つ

AIベースライン時代の品質管理

textlint-rule-preset-ai-writing :

AI生成文章の不自然さを検出

従来（～2022年）

人間には負担が大きい

AI時代（2023年～）

AIは瞬時に適用可能
一貫性の自動維持

textlintまとめ

技術的依存を増やした：

- AST拡張（構文解析ベース）
- ルール分離（100以上のエコシステム）
- MCP適応（AI時代への統合）

心理的依存を減らした：

- 全対応期待（No core rules戦略）
- コア肥大化懸念（エコシステムに委ねる）

Part 3

伝える技術

JavaScript Primer

書くから伝えるへの移行

書く過程で浮かび上がった『読者視点』が、伝える技術の枠組みを生み出した。

転換点：

- 個人で書く → 相手に伝える
- 一人で書く → 共著で書く
- **「読者」を強く意識する瞬間**

2016年：JavaScript Primer開発開始

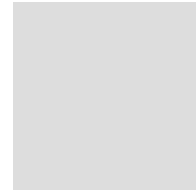
jsprimerの本質

3つを継続的に提供：

1. **書き方**：基本文法などのJavaScriptの書き方
2. **作り方**：アプリケーションの作り方
3. **学び方**：JavaScriptの進化を自分で知る学び方

JavaScriptのアンカー（基準点）

Living Standard戦略



Living Standard vs Snapshot

教科書もLiving Standardであるべき

- v6.0.0 : ES2024対応 (2024年)
- v7.0.0 : ES2025対応 (2025年)

Design Doc

各章/

└─ README.md (本文)

└─ OUTLINE.md (設計文書)

└─ 議論の履歴 (Issue/PR/Meeting Notes)

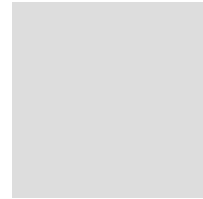
OUTLINE.mdの役割：

- なぜこの章が必要か
- 何を学ぶか
- どう教えるか

すべての議論を公開：

- Issue/PRでの議論
- Meeting Notesも公開

AI活用 - Iterator Helpers章



Design Doc to AI to Review

文章の「最初の一步」は重い

→ このボトルネックをAIで軽減

2025年時点での持続可能性の進化

既知→未知の原則

✖ 著者視点

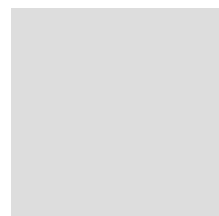
「これを覚える」

✔ 読者視点

「あなたが知っているXは、
実はY」

読者の現在地から出発

自動テストによる品質保証



3 Layer Quality Check

すべてCI/CDで自動実行

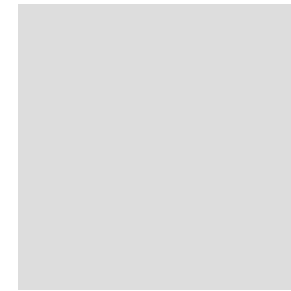
使っているから続く、続いているから使う

Sandpack統合（2022年）

ブラウザ内ライブ実行：

- コード例をその場で実行
- Contribute障壁の最小化
- GitHubだけで編集完結

技術的ハードルの低下 → より多くの貢献者



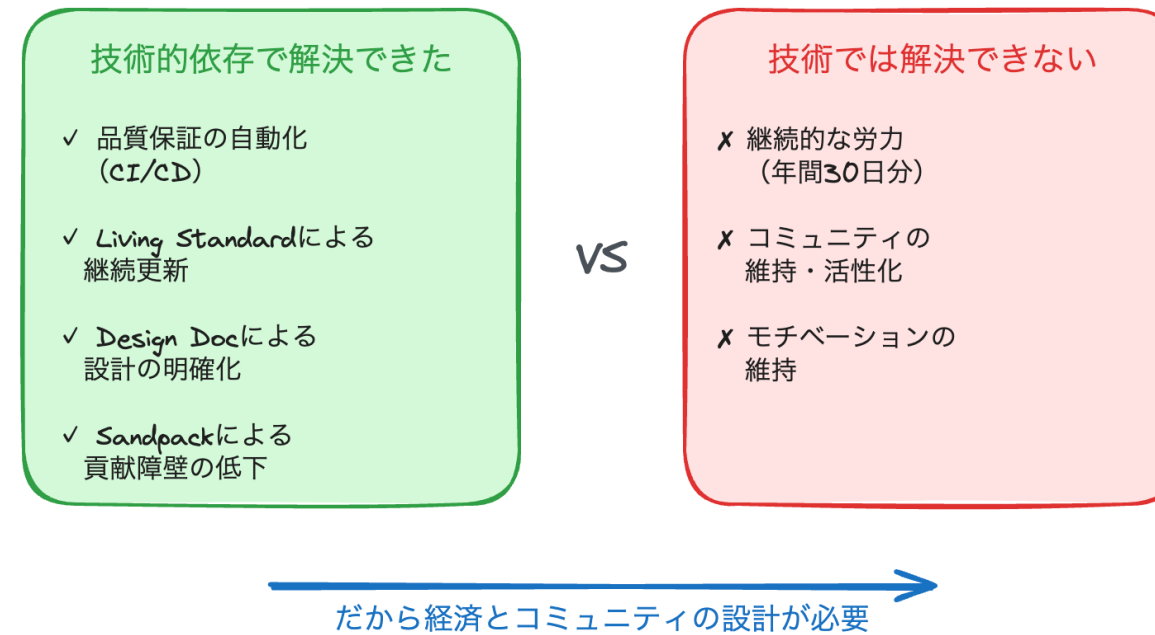
Live Code Execution

技術から人へ - 持続可能性の課題

技術的依存で解決できたこと：

- 品質保証の自動化（CI/CD）
- Living Standardによる継続更新
- Design Docによる設計の明確化
- Sandpackによる貢献障壁の低下

技術だけでは解決できない



技術的依存では解決できないこと：

- 継続的な労力（年間30日分）
- コミュニティの維持・活性化
- モチベーションの維持

→ だから経済とコミュニティの設計が必要

透明な経済モデル

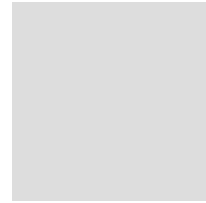
年間コスト試算：

- 約30日分の労力
- 約70万円相当

収入源：

1. 書籍売上
2. Open Collective
3. GitHub Sponsors

Open Collective



Open Collective Dashboard

仕組み：

- プロジェクト単位の資金管理
- 企業向けスポンサー特典
- 透明な収支公開

Fibonacci報酬システム

タスク複雑度：1， 2， 3， 5， 8ポイント

1ポイント = 約\$7

年間想定：60ポイント分の作業

革新性：

- 貢献度の可視化
- 金銭的還元の仕組み化

100人以上のコントリビューター

どうやって？

- ハードルを下げる
 - OUTLINE.mdで方向性明確化
 - Sandpackでブラウザ完結
- 透明性を保つ
 - すべての議論をGitHub上で
 - Meeting Notesも公開
 - 判断理由の記録

jsprimerまとめ

技術的依存を増やした：

- 継続仕様（Living Standard、ES2025対応）
- 自動品質（CI/CD、3層チェック）
- 設計文書（OUTLINE.md、AI活用）
- 経済透明化（Fibonacci報酬、Open Collective）

心理的依存を減らした：

- 一度で完成要求（継続更新前提）
- 単独著者責任（100人以上のコントリビューター）

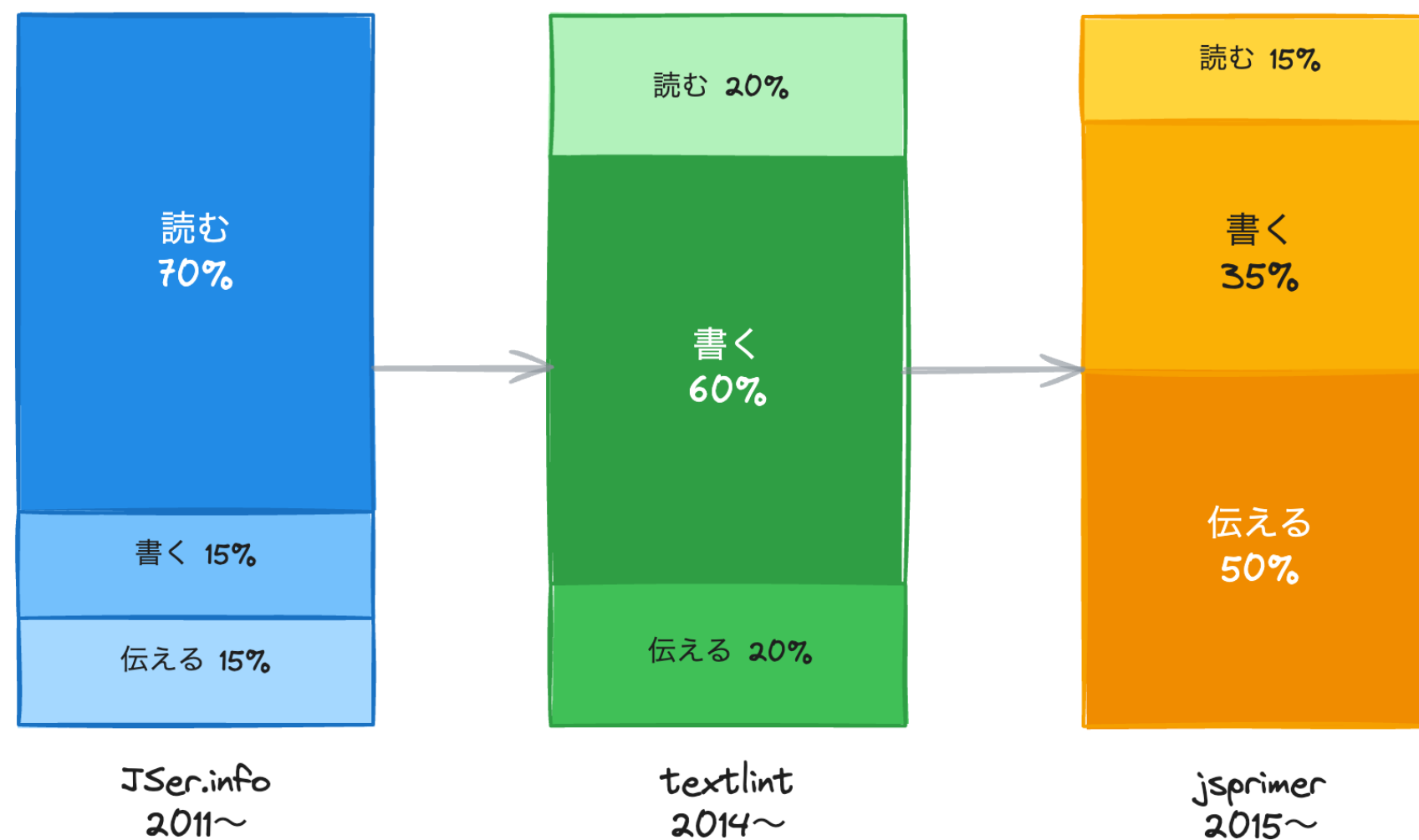
Part 4

循環の技術

複利効果と段階的發展

各プロジェクトの重点配分

読む技術・書く技術・伝える技術



段階的な発展

2011: JSer.info

読むことから始まった



2014: textlint

書くことで読む量が増える体感



2015: JavaScript Primer

伝えることで読む・書くの質が上がる

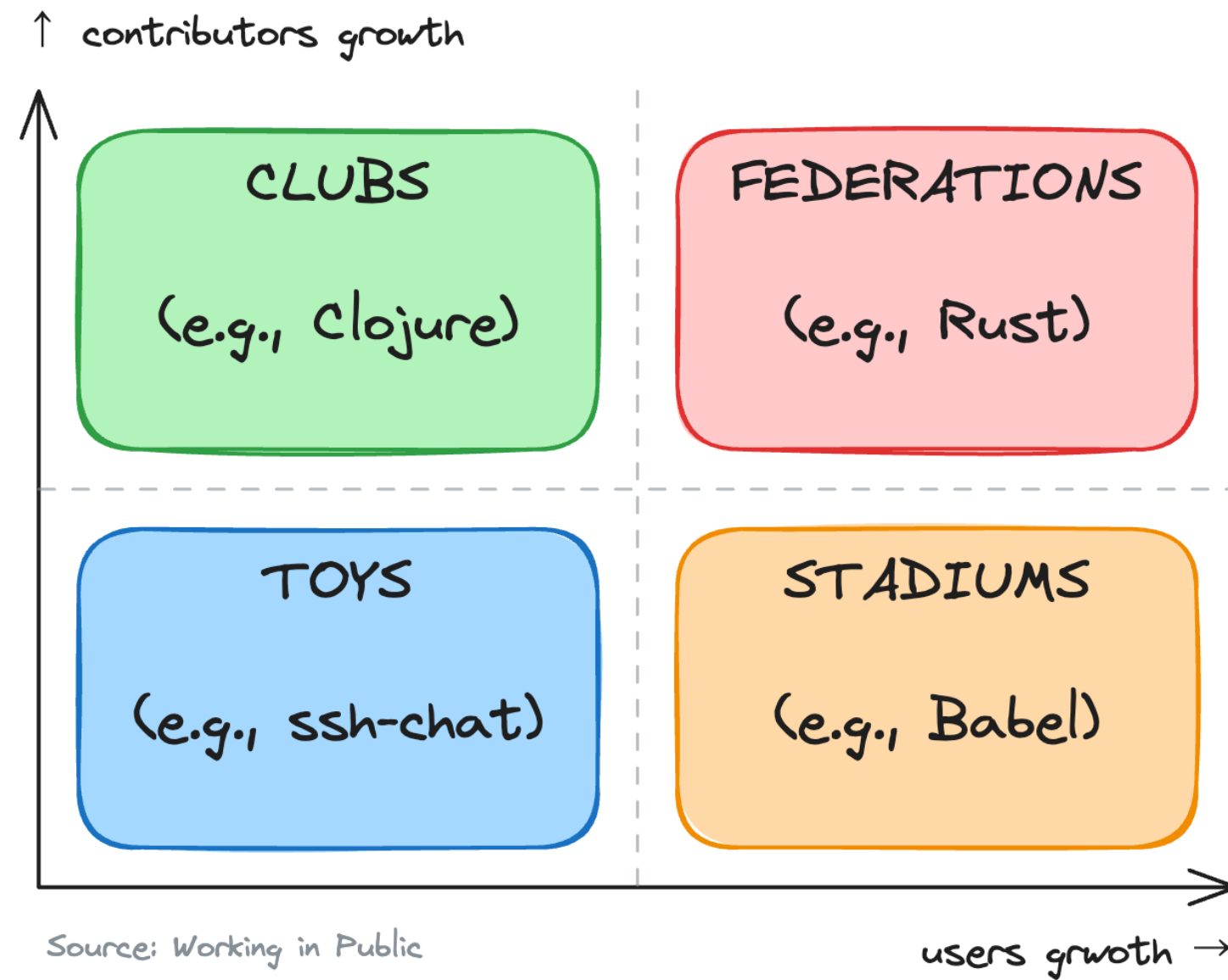
自然な進化、意図的な設計ではない

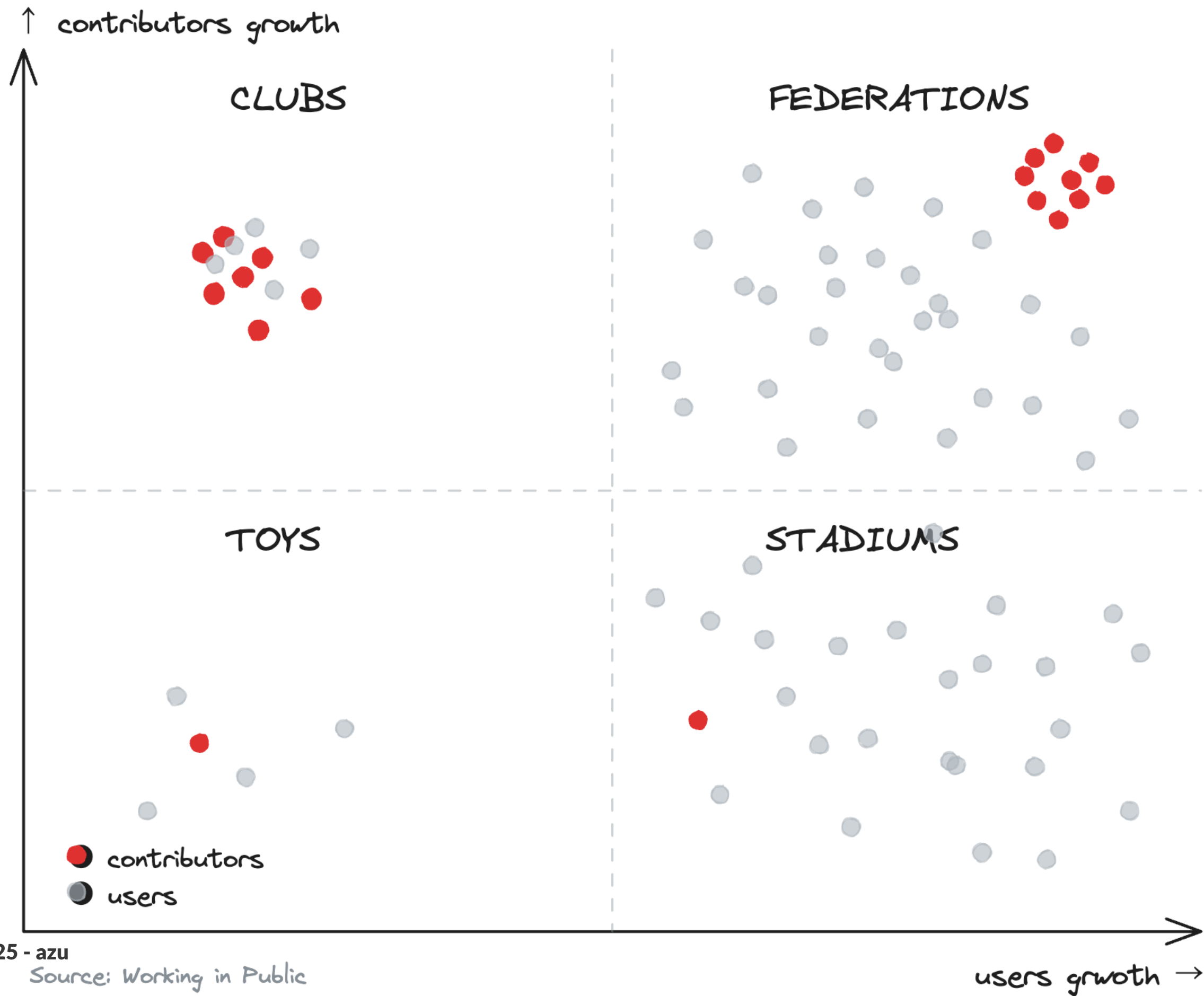
オープンソースの4つのモデル

モデル	ユーザー	貢献者	特徴
TOYS	Low	Low	サイドプロジェクト
STADIUMS	High	Low	集中型メンテナンス
CLUBS	Low	High	重複コミュニティ
FEDERATIONS	High	High	複雑なガバナンス

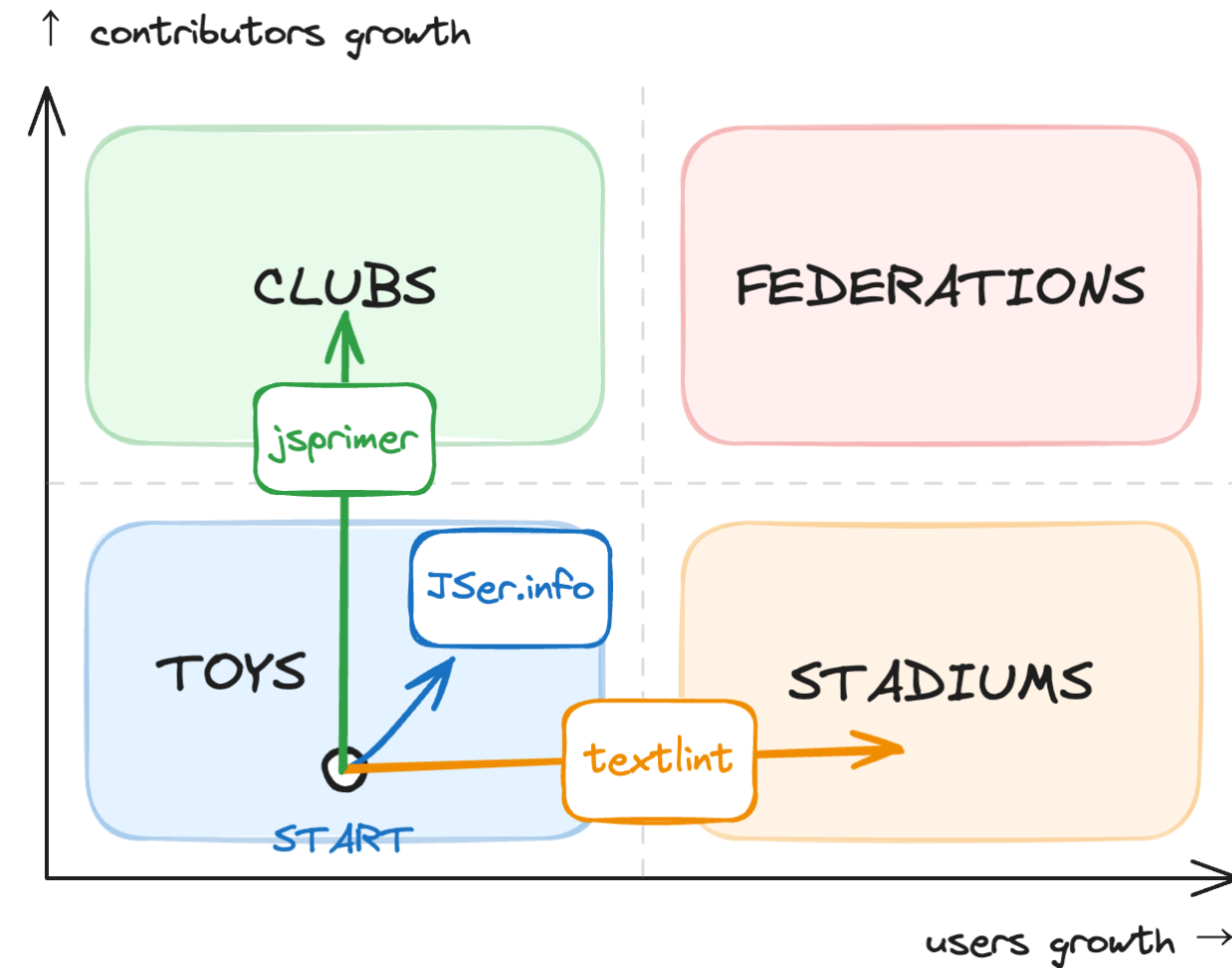
出典: [Working in Public](#)

4つのモデルの図





すべてのプロジェクトはToysから始まる



それぞれ異なるモデルへと発展

JSer.info: Toysモデルを維持

最適な規模を保つ設計：

- データドリブンな公開（GitHub Actionsによる自動化）
- 人間の判断を排除
- 心理的プレッシャーを発生させない設計

安定した規模を維持することで、心理的負荷を排除

textlint: Stadiumsモデル

週85,000ダウンロード、少数のメンテナー：

- No core rules戦略
- コアを最小化、エコシステムへ委譲
- 100以上のルールをコミュニティが作成

技術的依存は高いが、心理的依存を下げる設計

jsprimer: Clubsモデル

100人以上のコントリビューター：

- Living Standard（常に更新）
- Design Docで貢献のハードルを下げる
- ユーザーと貢献者が重複するコミュニティ

貢献しやすい構造で心理的負荷を分散

各モデルでの心理的負荷への対応

各モデルでの心理的負荷への対応戦略

clubs (左上: コントリビュータ多)

- 貢献のハードルを下げる
- Design Doc/Living Standard
- コミュニティの自律性を促進

Federations (右上: 高成長)

- ワーキンググループによる分散
- RFC/技術評議会による意思決定
- 責任を組織的に分散

Toys (左下: サイドプロジェクト)

- 成長より安定を優先
- データドリブン/自動化で負荷排除
- 心理的依存を発生させない設計

Stadiums (右下: ユーザー多)

- コアを最小化 (例: No core rules)
- エコシステムへの委譲
- メンテナーの負荷を技術的に削減

モデルごとに異なる戦略で持続可能性を確保

複利効果①データセット蓄積

JSer.info :

- 14年分のデータセット
- AI時代のサマライズ

textlint :

- 100以上のルール
- エコシステムの成長

jsprimer :

- Sandpack統合

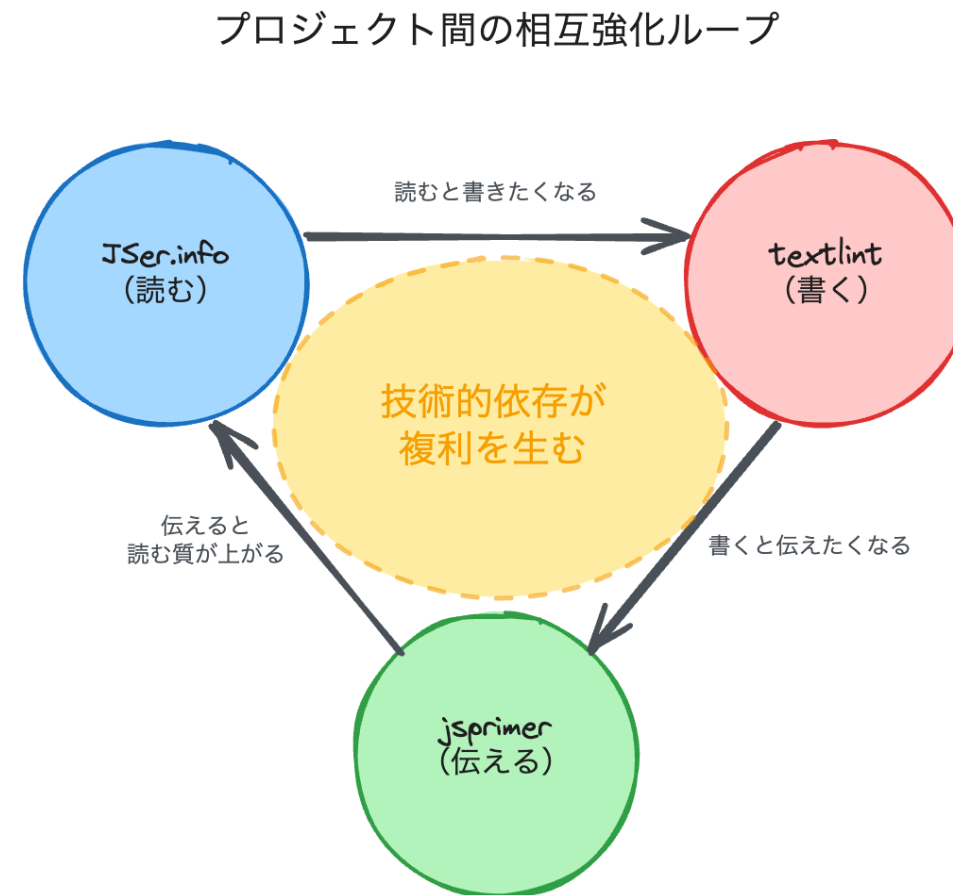
複利効果②AI時代への適応

続けていたから変化を取り入れられた：

- JSer.info MCP
- textlint MCP
- jsprimer: Design Doc + AI Agent

ゼロからではなく、蓄積を活かせる

複利効果③相互強化のループ



プロジェクト間の循環：

読む → 書く → 伝える → 読むの質向上

リスク - 技術的依存の連鎖

どれか一つが止まると全部止まる可能性

これは受け入れているリスク

トレードオフの認識

でも...

受け入れたリスクにもかかわらず、継続を支えた決定的な差異がありました。

リスクに対する設計上の緩衝

リスクは残るが影響度を小さく保つ設計：

- **再起性:** 復旧手順と自動化により中断から再開しやすい
- **分散:** No core / コントリビューター拡張で責任集中を回避
- **透明性:** 設計文書と経済公開で意思決定の負担を軽減

→ リスクは残るが影響度を小さく保つ設計で継続性を確保

技術的依存と心理的依存

技術的依存 ✓

- スケールする
- 疲れにくい
- 複利効果を生む
- 自動化できる
- **持続がし易い**

心理的依存 ✕

- スケールしない
- 疲れる
- バーンアウトを生む
- 自動化できない
- **持続が難しい**

具体例①JSer.info

データドリブン公開：

- 「毎週月曜」というプレッシャー排除
- データ量で判断
- 復旧可能性の向上

心理的依存を技術的依存に置き換え

具体例②textlint

No core rules :

- 「全てに対応する」プレッシャー排除
- エコシステムに委ねる
- 持続可能なメンテナンス

責任をコミュニティに分散

経済的持続可能性 - GitHub Sponsors

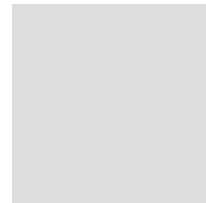
2019年10月から開始

105人以上のスポンサー

約¥2.59M/年 (\$17,300)

複数のプロジェクトを横断的に支援

GitHub Sponsorsの成長



2019-2024 Sponsor Growth

2019年10月：\$0（開始）

2024年：約¥2.59M/年（\$17,300）

なぜ成長できたのか？

GitHub Sponsorsの設計思想

Tier設計：

- ✨ Supporter：\$1/月
- ☕ Coffee：\$5/月
- 🌐 Domain：\$10/月
- 📖 Book：\$30/月

特典は最小限：

責任感・義務感を作らない

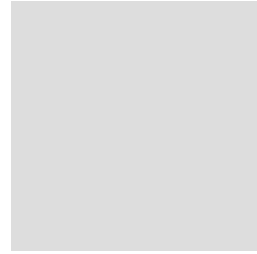
具体例③GitHub Sponsors

見返りを用意しない：

- 即時利益より長期関係
- 変に義務を作るとプレッシャーになる
- 健全な支援関係

義務感を作らない設計

Public Writing as Future Asset



15years Public Writing Cycle

この循環こそが「書くことは読むことである」の実証

ま

と

め

12の設計原則

読む技術 (JSer.info) :

1. データドリブンな公開基準 [技術]
2. 退屈な部分だけ自動化 [技術]
3. 小さなイテレーション [技術]

書く技術 (textlint) :

4. デフォルトルールなし戦略 [心理]
5. 最小コア、豊かなエコシステム [技術]
6. 検証可能なことを自動化 [技術]
7. 最小限の依存関係 [心理]

伝える技術 (jsprimer) :

8. Living Standard戦略 [心理]
9. 既知→未知の原則 [心理]
10. 自動テストによる品質保証 [技術]
11. 透明な経済モデル [心理]
12. 使って改善する循環 [技術/心理/循環]

[技術]: 技術的依存を育てる (1,2,3,5,6,10,12)

[心理]: 心理的依存を減らす (4,7,8,9,11,12)

続けることで複利が生まれる

- データセット蓄積
- エコシステム成長
- AI時代への適応力
- プロジェクト間の相互強化

そのための設計原則：

- **技術的依存を積極的に育てる** → スケールする、疲れにくい、複利効果
- **心理的依存を意識的に減らす** → バーンアウト回避、長期継続

ありがとうございました

- JSer.info: <https://jser.info/>
- textlint: <https://textlint.org/>
- JavaScript Primer: <https://jsprimer.net/>
- GitHub: <https://github.com/azu>

